Deep Learning in Computer Vision· Lecture 3 of 9

Generalization, Training Problems & Evaluation

Overfitting, class imbalance, vanishing gradients, metrics

Training Problems: Overfitting

- **Overfitting** just means that **your trained model predicts well on the data that you used to train it, but performs poorly in the real world on new data it hasn't seen before**
  - The model learns the training data **too well**, even its noise
    - This can happen if one parameter is weighed too heavily and ends up dominating the formula
  - High training accuracy, **poor accuracy on unseen data**
  - Like memorizing exam answers instead of understanding
- Detect it: **validation loss rises** while training loss keeps falling
- Use separate **train / validation / test** sets

Avoiding Overfitting: Regularization

- Regularization is a term added into the optimization process that helps avoid overfitting.
- In regularization, a special piece is added onto the loss function that penalizes large weight values to avoid overfitting.
  - **Add a penalty on large weights to the loss**
    - Total Loss $L = L_{\text{data}}$(primary loss calculated on your training data) + **Penalty Term**
      - **L1 Regularization**: L = L_data + **λ Σ |w|** (sparse weights)
      - **L2 Regularization**: L = L_data + **λ Σ w^2** (weight decay)
      - **λ** (Regularization Strength) controls the strength of the penalty
        - Usually a small positive value like 0.01or 0.001
      - Keeps one weight from dominating the prediction

Other Popular Regularization Techniques:

Dropout and Early Stopping (Loss Equation L is NOT modified)

Dropout

During training **randomly switch off** a fraction *p* of neurons each step (e.g. p = 0.5).

Forces redundancy; acts like averaging many sub-networks.

Dropout is turned OFF during testing/inference. All neurons are used.

Early stopping

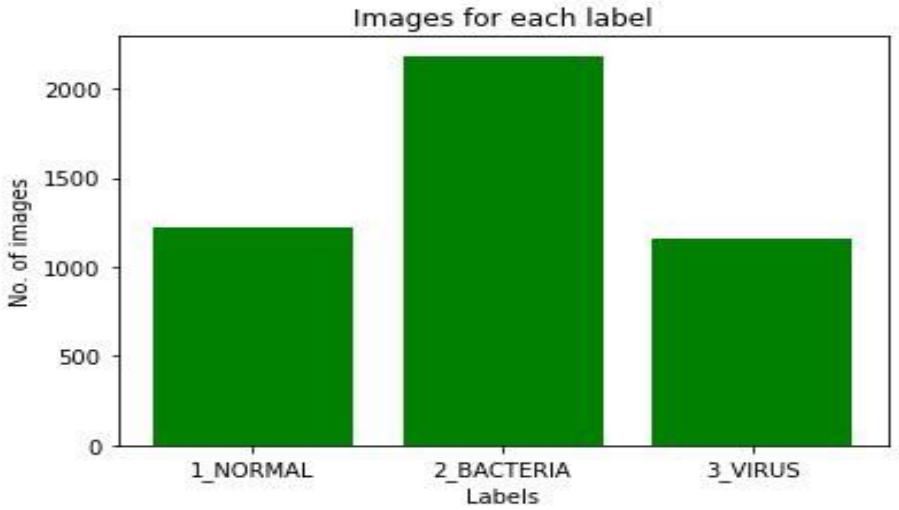
Watch the **validation loss**; stop when it no longer improves for a few epochs and keep the best weights.

Free and very effective.

Training Problems: Class Imbalance Problem

- Class imbalance in computer vision happens when some categories (classes) in a dataset have far fewer images than others.
- The number of examples that belong to each class may be referred to as the class distribution.
- Imbalanced classification refers to a classification predictive modeling problem where the number of examples in the training dataset for each class label is not balanced.

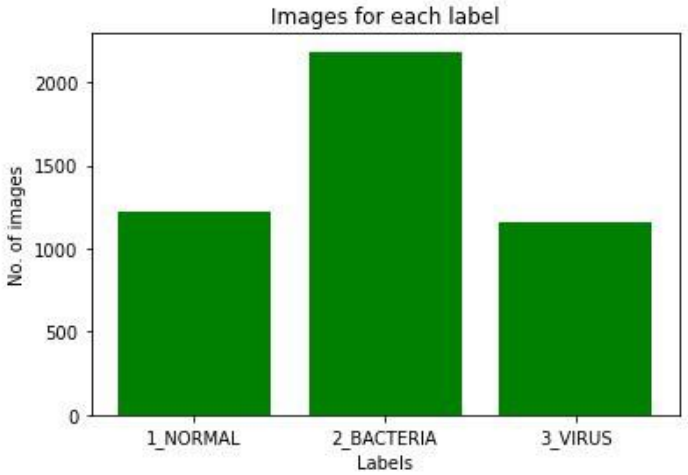
```
Number of Normal images = 1226
Number of Bacteria images = 2184
Number of Virus images = 1154
```



Class Imbalance

- Some classes have **far fewer** images than others
- Ratio 1:100, or 80% / 18% / 2%
- Slight (4:6) is fine; **severe (1:100)** needs special handling
- Model is biased to the **majority class**
- The minority class is usually the **important** one (tumor, fraud, defect)

```
Number of Normal images = 1226
Number of Bacteria images = 2184
Number of Virus images = 1154
```

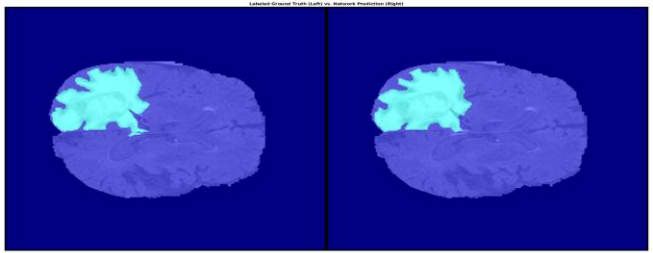


Majority/ Minority classes

- **Majority Class**:
  - The class that is overrepresented (has many examples) in an imbalanced dataset, potentially leading to biased models.
    - Example: Think of a classification predictive modeling problem
      - Classifying whether an incoming email is "Spam" or "Not Spam".
- **Minority Class**:
  - The class in an imbalanced classification predictive modeling problem that has few examples.
- **The learning process of most classification algorithms is often biased towards the majority class examples, so that minority ones are not well modeled**
- In real life problems minority class is usually more important

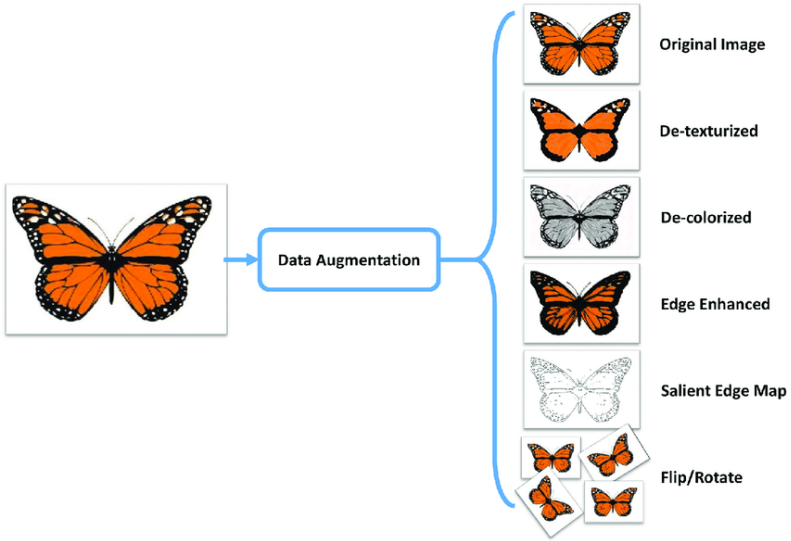
In a classification problem of “Cat” and “Dog”, (1000 samples for cats, 10 samples of dog), in this case:

- Cat is majority class
- Dog is minority class



Treating Class Imbalance Problem:
Data Augmentation

- Create new training images by **transforming** existing ones
  - Flip, rotate, translate, crop
  - Blur, brightness, contrast, noise
  - Adds variety → **less overfitting**
  - Also helps with minority classes



Dealing with Class Imbalance

Data level

Undersample majority (loses information)

Oversample minority (risk of overfitting)
Synthetic samples (e.g. SMOTE): Synthetic Minority Over-sampling Technique (SMOTE) generates new, non-identical minority samples by interpolating between neighboring feature points in feature space

Augmentation

Transform minority images (flip, rotate, brightness) to add **new** samples.

Algorithm level

Class weights / cost-sensitive loss
Assign a higher penalty weight in the loss function to errors made on minority samples.

Focal loss
Designed specifically for extreme class imbalance (e.g., object detection), Focal Loss dynamically down-weights the loss assigned to easy-to-classify, confident majority samples. It focuses the backpropagation gradients almost entirely on hard, unconfident, or rare minority instances

Vanishing Gradient Problem

- Vanishing Gradient Problem occurs when, during training, the gradients (small values used to adjust weights) become so tiny that they stop changing the weights meaningfully, slowing down or even stopping learning, especially in deep layers.

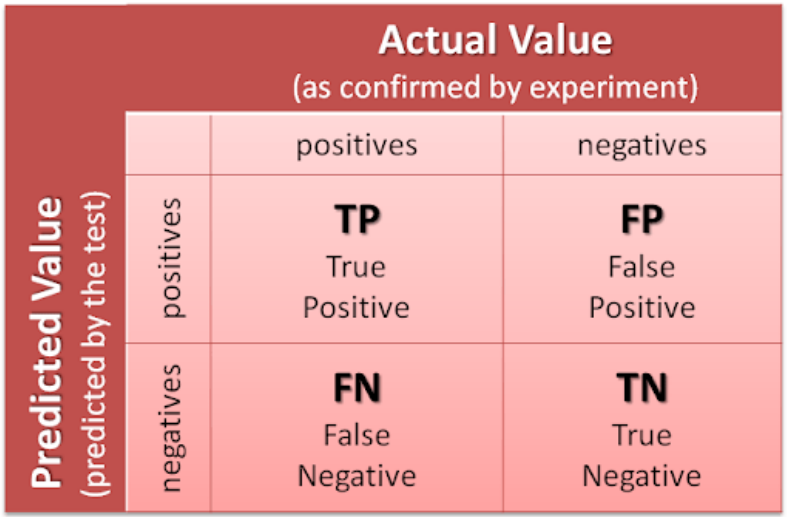
- **Example:** Picture trying to push a heavy door open but only being able to exert a tiny, nearly ineffective force. No matter how many times you push, the door barely moves.

- Fixes: **ReLU**, good initialization, **batch normalization**, **skip connections** (ResNet, Lecture 6)

Evaluation Metrics for DL

Base measures

- **True Positive (TP):** A true positive is an outcome where the model *correctly* predicts the positive class.
- **True Negative (TN):** A true negative is an outcome where the model *correctly* predicts the negative class.
- **False Positive (FP):** A false positive is an outcome where the model *incorrectly* predicts the positive class.
- **False Negative (FN):** A false negative is an outcome where the model *incorrectly* predicts the negative class.



Confusion Matrix

Evaluation Metrics

- **Evaluation metrics are tied to machine learning tasks**. There are different metrics for the tasks of classification, regression, ranking, clustering, topic modeling, etc.

- **Example**

  - **Accuracy =** $\frac{TP+TN}{TP+TN+FP+FN}$
  - **Precision =** $\frac{TP}{TP+FP}$
  - **Recall =** $\frac{TP}{TP+FN}$
  - **F1-Score =** $\frac{2\times Precision \times Recall}{Precision + Recall}$

Predicted Value (predicted by the test)		Actual Value (as confirmed by experiment): positives	Actual Value (as confirmed by experiment): negatives
	positives	**TP** True Positive	**FP** False Positive
	negatives	**FN** False Negative	**TN** True Negative

Confusion Matrix

Evaluation: The Confusion Matrix

Example: Building a prediction model for hospitals to identify whether the patient is suffering from cancer or not . Assume: 100 patients, 10 have cancer.

Trained Model	Actual: Cancer	Actual: Normal
Predicted: Cancer	TP = 1	FP = 1
Predicted: Normal	FN = 9	TN = 89

Accuracy = (TP+TN) / all = 90%
Looks great, but a model that says "Normal" every time also scores 90%.

TP (**Actual** Cancer and predicted Cancer) = **1**
TN (**Actual** Normal and predicted Normal) = **89**
FN (**Actual** Cancer and predicted Normal) =**8**
FP (**Actual** Normal and predicted Cancer) = **1**

Accuracy is misleading on imbalanced data.

Precision, Recall and F1

Trained Model	Actual: Cancer	Actual: Normal
Predicted: Cancer	TP = 1	FP = 1
Predicted: Normal	FN = 9	TN = 89

Precision

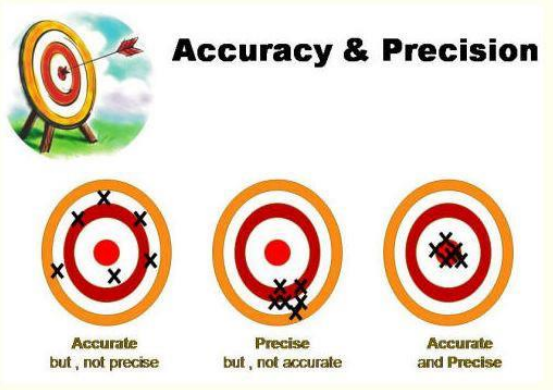
TP / (TP+FP)
= 1/2 =
0.50
"Of predicted positives, how many are right?"

Recall (TPR)

TP / (TP+FN)
= 1/10 =
0.10
"Of real positives, how many found?"

F1 score

2PR / (P+R)
=
0.17
Harmonic mean; punishes extremes

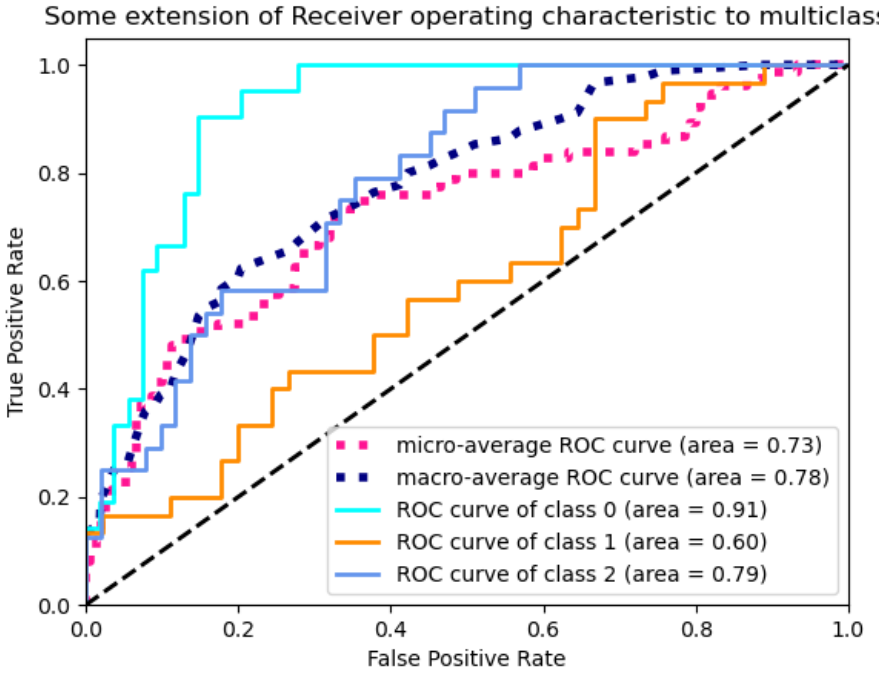


Precision, Recall and F1

- Unfortunately, precision and recall are often in **conflict**.
    - That is, improving precision typically reduces recall and vice versa.
- So, whether high precision or high recall is preferred would depend upon the actual domain/use case, but there exists another evaluation metric known as an F1 Score that helps us get the best of both metrics.
    - Example: Screening favours recall;
    - Example: Spam filters favour precision.

ROC Curve

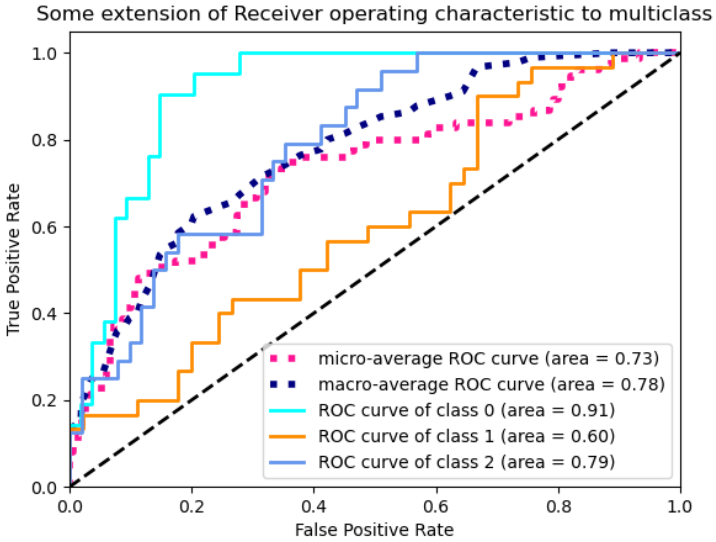
- A ROC curve is a graphical representation of the performance of a binary classification model for all classification thresholds.
- Here, *ROC* stands for Receiver Operating Characteristic.
- It can be used also to compare between different classifiers



ROC Curve and AUC

- Classifiers output a score; the **threshold** decides positive/negative
- Sweep the threshold: plot **TPR** = TP/(TP+FN) vs **FPR** = FP/(FP+TN)
  - **TPR**: True Positive Rate
  - **FPR**: False Positive Rate
- Diagonal = random guessing
- **AUC** = area under the curve: 1.0 perfect, 0.5 random
- Compares classifiers regardless of threshold

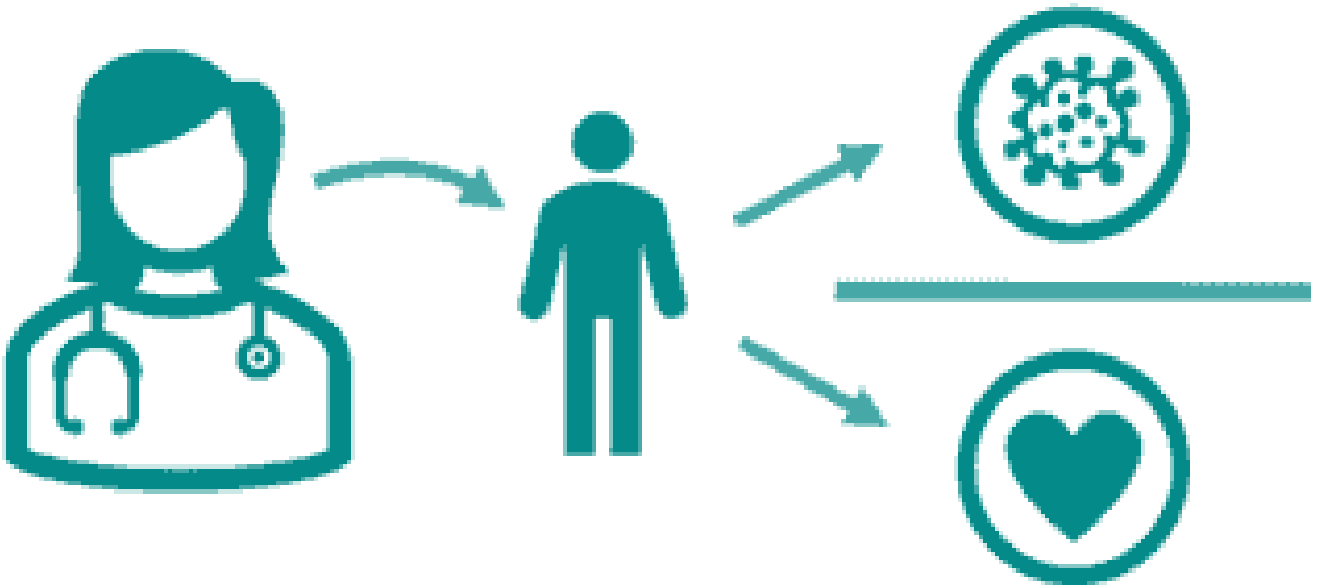
		Actual Value (as confirmed by experiment)	
		positives	negatives
Predicted Value (predicted by the test)	positives	**TP** True Positive	**FP** False Positive
	negatives	**FN** False Negative	**TN** True Negative



Week 1 Summary

- Neurons → fully connected nets → **training with backprop + optimizers**
- Control overfitting: **regularization, dropout, augmentation, early stopping**
- Handle imbalance and vanishing gradients
- Evaluate with **precision, recall, F1, ROC/AUC**, not accuracy alone
- Week 2: **CNNs**, the architecture built for images

Additional Slides (Required but Self reading)

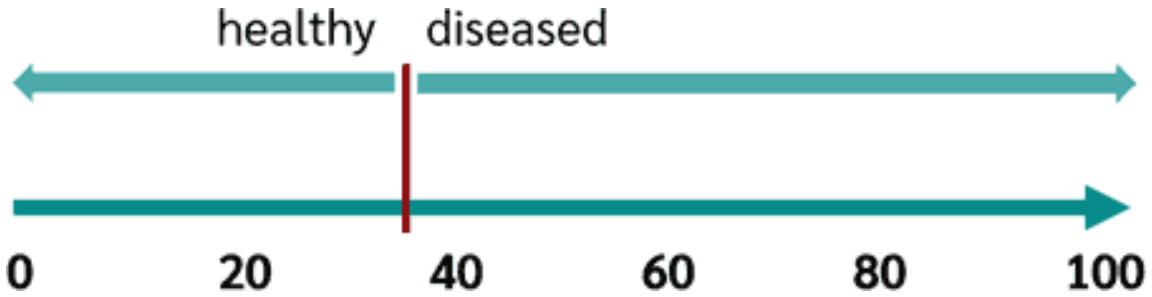


Example of a ROC curve

- **We would like to classify, based on a screening, whether a person has cancer or not.**

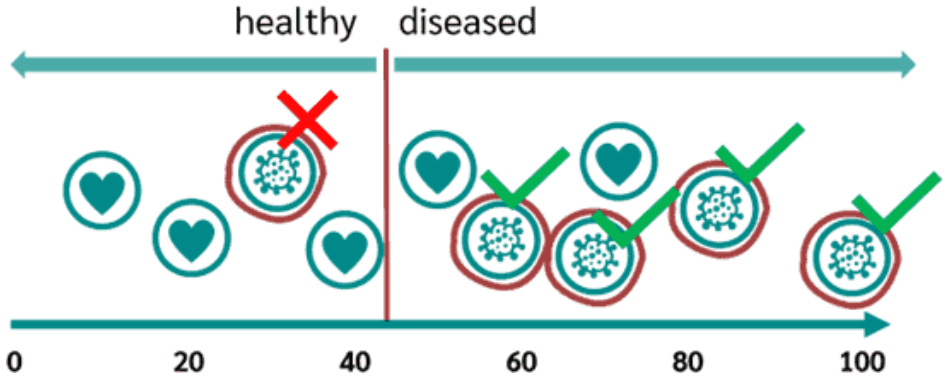
Example of a ROC curve

- This **classification** is done with the **help of a certain blood value**, where high values indicate cancer.
- The question now is **which value we choose as the classification threshold.**
  - So from **which value do we predict a disease?**



Example of a ROC curve

- For this, we **obtain data** say from **10 people** about:
  1. how high the blood value is and
  2. whether or not the disease is present.
- We could now **choose a classification threshold of 45**, for example.
  - In this case, of the 5 people with the disease, we would correctly classify 4 as "diseased" and 1 incorrectly as "healthy."
  - Thus, we correctly classified 4 out of 5 as "diseased."





True Positive Rate and False Positive Rate

- The True Positive Rate or sensitivity or recall is equal to the true positives divided by the true positives plus the false negatives. The true positives are those correctly classified as "diseased" and the false negatives are those incorrectly classified as "healthy".

$$\textbf{True Positive Rate} = \frac{\textit{True Positives}}{\textit{True Positives} + \textit{False Negatives}}$$

Correctly classified as "diseased“

Incorrectly classified as "healthy“

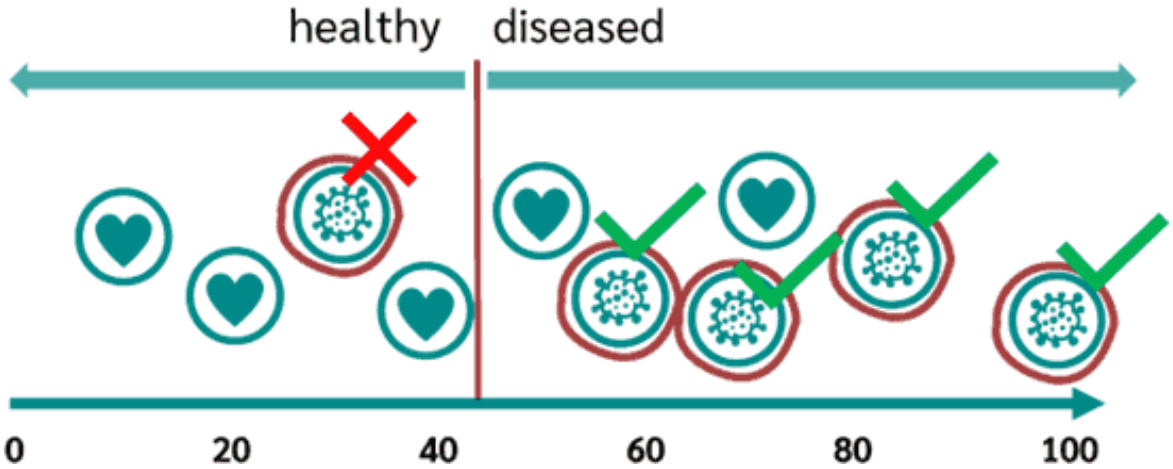
False Positive Rate (FPR)

- The **False Positive Rate** or **Specificity** is equal to the false positives divided by the false positives plus the true negatives. The false positives are the healthy individuals misclassified as "diseased" and the true negatives are the individuals correctly classified as "healthy".

$$\textit{False Positive Rate} = \frac{\textit{False Positives}}{\textit{False Positives} + \textit{True Negatives}}$$

Healthy persons misclassified as "diseased"

Correctly classified as "healthy"

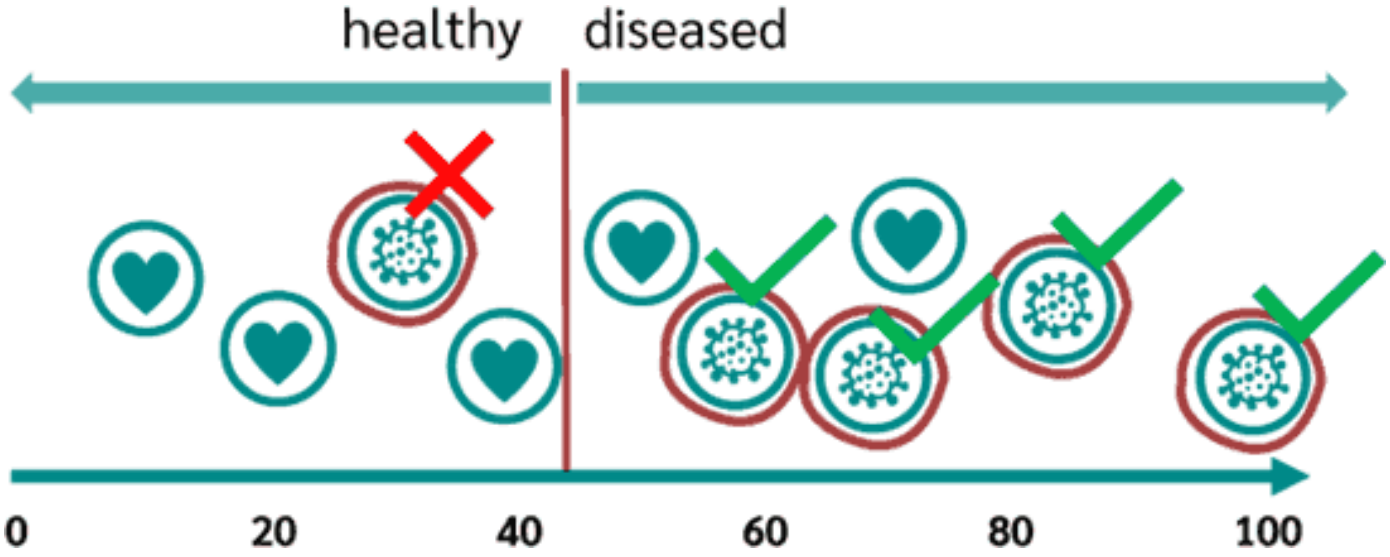


- Recall Rate (also called true positive rate TPR or **sensitivity**) measures the proportion of positive instances that are accurately detected as positive by the model. It is calculated as:

- **Recall = TP / (TP + FN) = 4 / 5**

  - TP (True Positive) – The *positive* instances *correctly* classified.

  - FN (False Negative) – The *negative* instances *incorrectly* classified.



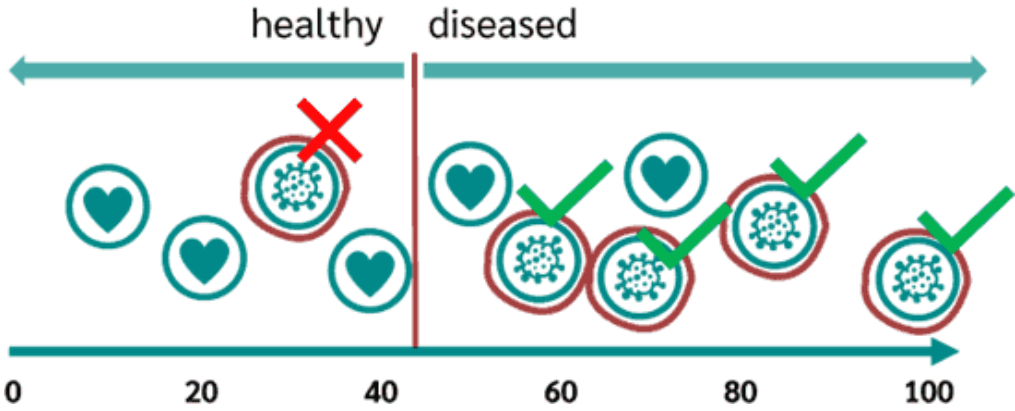
- **False Positive Rate (FPR) or specificity.**
  - out of the 5 healthy individuals, we have 2 misclassified as "diseased" and 3 correctly classified as "healthy". So, we misclassified 2 out of 5 as "diseased". This value is called FPR

Summary:

Calculate TPR (Recall or Sensitivity) & FPR (Specificity)

Predicted Value (predicted by the test)		Actual Value (as confirmed by experiment): positives	Actual Value (as confirmed by experiment): negatives
	positives	**TP** True Positive	**FP** False Positive
	negatives	**FN** False Negative	**TN** True Negative

- TPR = 4/(4+1) = 4/5 = 0.8
- FPR = 2/(2+3) = 2/5 = 0.4



$$\textit{True Positive Rate} = \frac{\textit{True Positives}}{\textit{True Positives + False Negatives}}$$

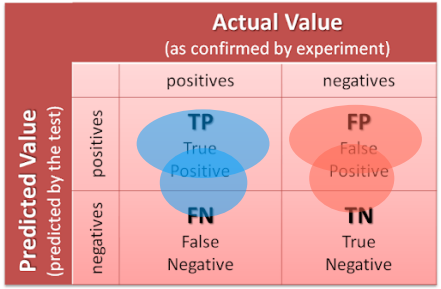
Correctly classified as "diseased"

Incorrectly classified as "healthy"

$$\textit{False Positive Rate} = \frac{\textit{False Positives}}{\textit{False Positives + True Negatives}}$$

Healthy persons misclassified as "diseased"

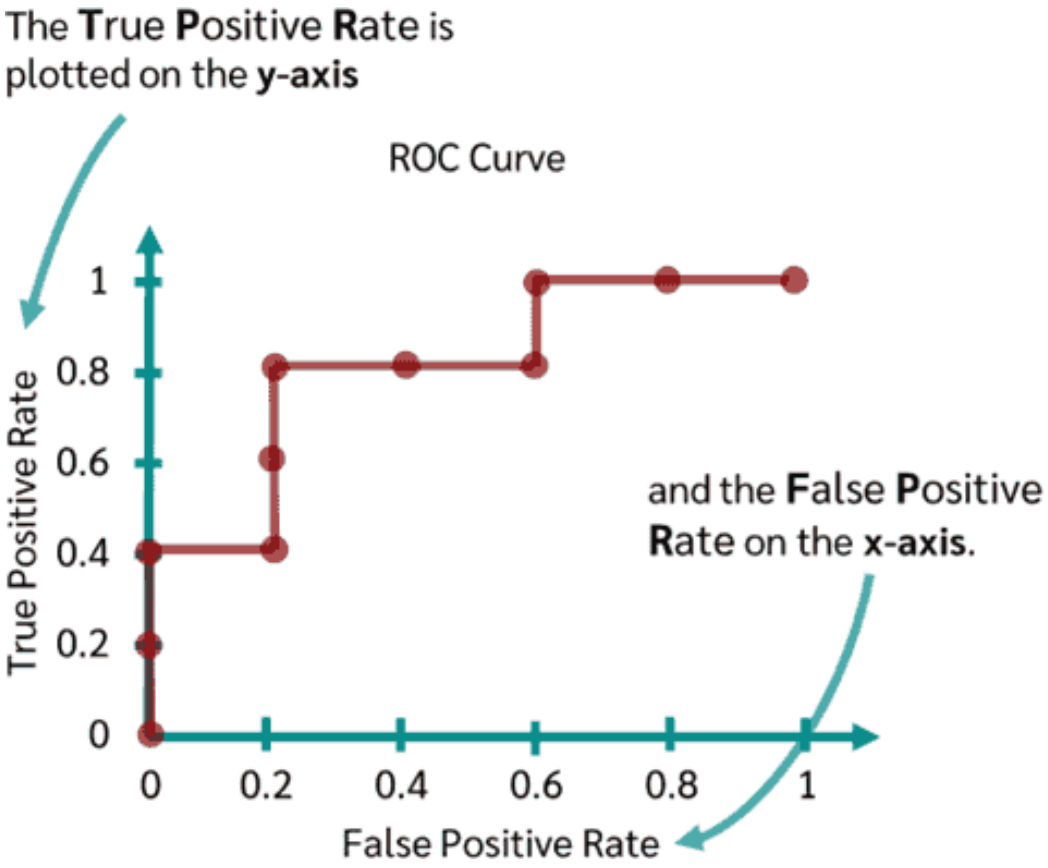
Correctly classified as "healthy"



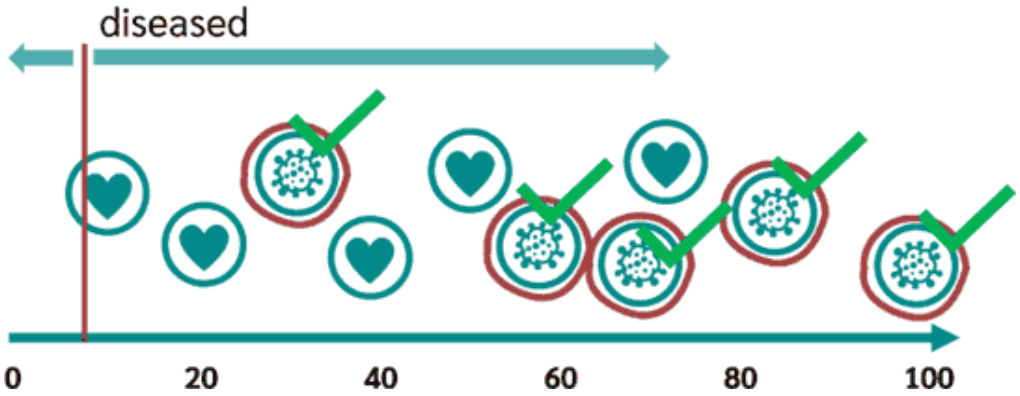
Plot the ROC Curve

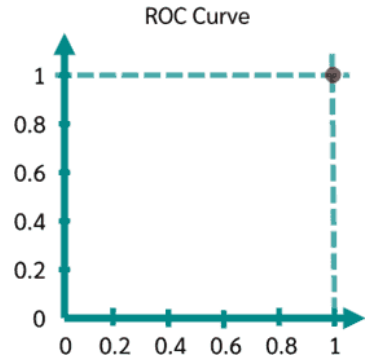
- We can now calculate for each threshold what the True Positive Rate and the False Positive Rate are. These two values are plotted on the ROC curve.

- The True Positive Rate is plotted on the *y-axis* and the False Positive Rate on the *x-axis*.



If we choose the threshold value to be very small

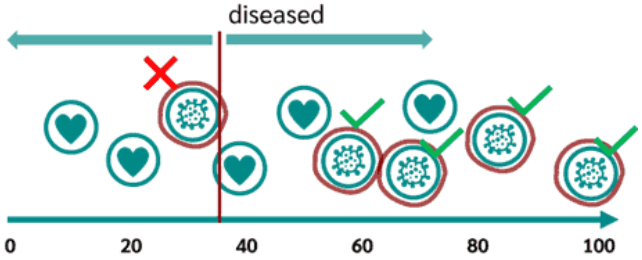




Predicted Value (predicted by the test)		Actual Value (as confirmed by experiment): positives	Actual Value (as confirmed by experiment): negatives
	positives	TP True Positive	FP False Positive
	negatives	FN False Negative	TN True Negative

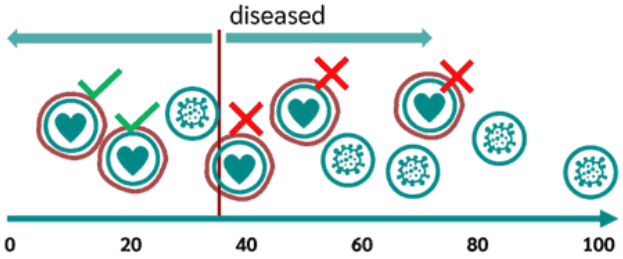
- TPR = 5/(5) = 1
- FPR = 5/(5) = 1

True Positive Rat

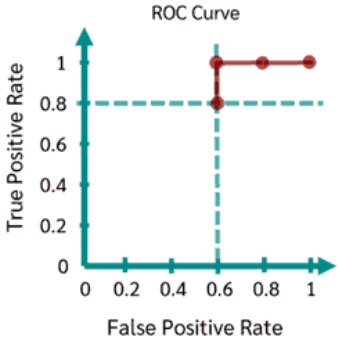


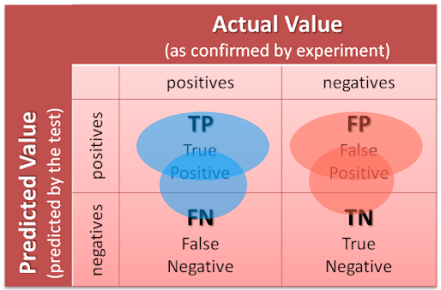
- TPR = 4/(4+1) = 4/5=0.8

False Positive Rat



- FPR = 3/(3+2) = 3/5 = 0.6





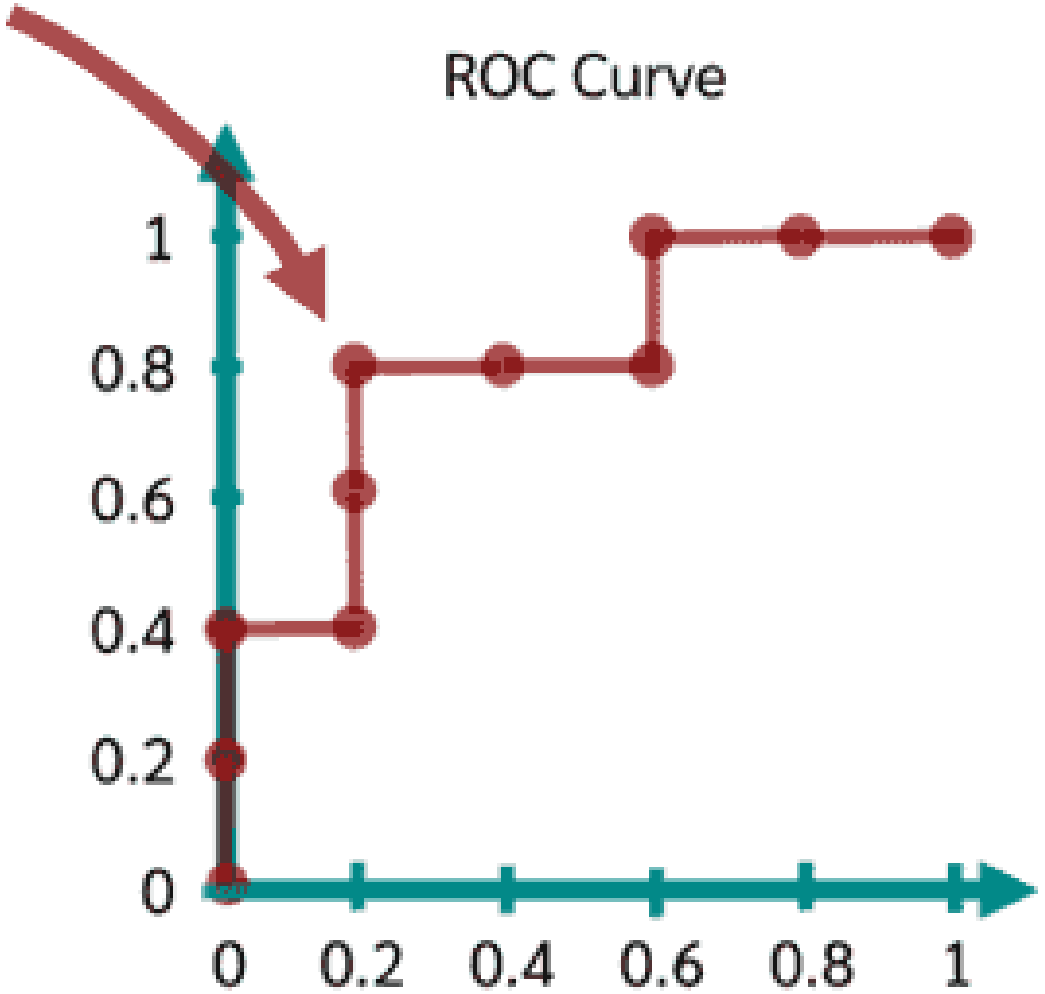
ROC Curve

- We can do this for all other thresholds, finishing the ROC curve.

- At **the marked point beside**, for example:

  - **80%** of the **diseased people** were **correctly classified as "diseased"** and

  - **20%** of the **healthy people** were **incorrectly classified as "diseased"**.



AUC Value

- Using the ROC curve, we can now also compare different classification methods.
- A classification model is better the higher the curve is.
- Therefore, the larger the area under the curve, the better the classifier.
    - Exactly this area, the Area under the Curve, is reflected by the AUC value.
- The AUC value varies between 0 and 1. The larger the value, the better the classifier.

